

Dual_FreqDAE 기술 이전 문서

1. 개요 (Context)

Dual_FreqDAE는 단일 심박(beat) 단위 ECG 신호(샘플 길이 512, 360Hz 기준)의 잡음을 제거하기 위한 이중 분기(dual-branch) 오토인코더입니다.

구성 요소:

- 시간(Time) 도메인 분기: Gated Conv1D 인코더 (sigmoid × linear/ELU 조합) + 배치 정규화.
- 주파수(Frequency) 도메인 분기: FFT Magnitude(절반 스펙트럼 복제) 기반 Conv1D 인코더.
- 결합(Fusion): 두 분기 latent concat + 위치 임베딩(Positional Encoding) + FANformer(멀티헤드어텐션 + FANLayer) 블록 반복.
- 디코더: 잔차(Residual)를 활용한 Conv1DTranspose 업샘플 체인으로 최종 재구성.

입력 형태: (512, 1) 한 비트. 출력 형태: 노이즈 제거된 비트 (512, 1). 목표: 베이스라인/광대역 혼합 노이즈 제거 + P-QRS-T 형태 보존.

2. 선행 요구사항 (Prerequisites)

항목	권장
하드웨어	CUDA 지원 GPU (예: RTX 시리즈) – CPU도 가능하나 느림
운영체제	Linux (Ubuntu) 또는 macOS
소프트웨어	Conda, CUDA Toolkit & 드라이버 (TensorFlow 호환 버전)

TensorFlow / CUDA / cuDNN 버전 불일치 시 학습 속도 저하 혹은 초기화 오류 가능 → 환경.yml 버전 고정 권장.

3. 환경 구축 (Environment Setup)

```
conda env create -f environment.yml
conda activate ecgdenoise
# 필요 시 environment.yml 수정하여 TF/NumPy 버전 고정
```

다른 CUDA 스택 사용 시: 설치된 드라이버와 TF 빌드 호환성(Compute Capability)을 확인하십시오.

4. 데이터 자산 (Data Assets)

필수 파일:

- data/QTDatabase.pkl: QTDatabase에서 추출된 Beat 단위 신호(360Hz 리샘플)
- data/CombinedNoise_Train.pkl: 학습용 노이즈 소스

- `data/CombinedNoise_Test.pkl`: 테스트용 노이즈 소스

선택(재생성용 원천):

- `data/qt-database-1.0.0/` 내 원천 PhysioNet QT 데이터

유효성 체크:

1. `QTDatabase.pkl` 로드 → `dict[record_id] = list[beat_np_array]`
2. 비트 길이 가변 → 파이프라인에서 512 길이 Zero-padding 처리
3. 노이즈 pickle 로드 시 shape 오류 없을 것

5. 운영 절차 (Operations)

5.1 학습 (Training)

```
python tools/train_dual_freqdae.py \
  --exp-dir experiments/run1 \
  --data-prep-samples 512 \
  --epochs 100000 \
  --patience 10 \
  --min-delta 0.05 \
  --reuse-cache
```

산출물(Artifacts):

- `Dual_FreqDAE_weights.best.weights.h5`: 최적 가중치 (`val_loss` 기준)
- `history.json`: 학습/검증 손실 및 메트릭 곡선
- `metrics_summary.json`: RMSE, PRD, COS_SIM, SNR 요약 통계
- `metrics_per_sample.csv`: 테스트 세트 각 비트별 메트릭

5.2 평가만 실행 (Evaluation Only)

```
python tools/train_dual_freqdae.py --exp-dir experiments/run1 --evaluate-only
```

5.3 드라이런 (Dry Run)

모델/데이터 준비 후 학습 생략:

```
python tools/train_dual_freqdae.py --dry-run --exp-dir experiments/dry
```

5.4 추론 (Inference)

배치 비트 추론 (형태: (N,512), (N,512,1)):

```
python tools/run_dual_freqdae.py \
  --input exported/X_test.npy \
  --weights experiments/run1/Dual_FreqDAE_weights.best.weights.h5 \
  --output denoised_X_test.npy
```

연속 1D 신호(긴 ECG)를 512 윈도우로 분할 후 추론:

```
python tools/run_dual_freqdae.py \
  --input long_signal.npy \
  --segment --hop 512 --pad \
  --weights experiments/run1/Dual_FreqDAE_weights.best.weights.h5 \
  --output denoised_segments.npy
```

6. 모니터링 & 품질 관리 (Monitoring & QA)

- `val_loss` 감소 흐름 + `ReduceLROnPlateau` 학습률(LR) 단계적 감소 확인 (종료 시 LR ~1e-6~1e-7 수준)
- `metrics_summary.json` 비교: 기존/베이스라인 대비 PRD ↓, COS_SIM ↑, SNR ↑ 여부 검증
- 시각적 품질: 랜덤 샘플 20개 정도 그려 원본 vs 노이즈 vs 복원 비교
- 이상 탐지: 특정 비트에서 RMSE 과도하게 높을 경우 원천 비트 길이/패딩 문제 재점검

7. 커스터마이징 (Customization)

목적	방법
Transformer 깊이 조정	<code>dl_models.py</code> 내 <code>num_transformer_blocks</code> 변경
숨김 차원/헤드 변경	<code>head_size</code> , <code>num_heads</code> , <code>hidden_dim</code> 파라미터 수정
손실 대체	<code>combined_ssd_mad_loss</code> → Huber/Frequency-domain/다중 목적 함수 적용
노이즈 주입 방식	<code>AddGatedNoise</code> → <code>ParametricNoiseInjection</code> 교체
혼합 정밀도	<code>train_dual_freqdae.py</code> 상단에 <code>tf.keras.mixed_precision.set_global_policy('mixed_float16')</code> 추가
속도 향상	Batch size 증가(메모리 허용), XLA 활성화 (<code>TF_XLA_FLAGS</code>)
경량화	추후 pruning/quantization 스크립트 추가 (TensorFlow Model Optimization)

8. 유지보수 & 인수인계 (Maintenance & Handover)

- 환경 고정: `environment.yml` 내 주요 패키지 버전(tensorflow, numpy, scipy) 고정 후 사내 아티팩트 레지스트리에 업로드.

2. **가중치 관리:** `experiments/` 디렉터리 하위 실험별 폴더 네이밍 규칙 수립 (예: `runYYYYMMDD_lr1e-3_block8`).
3. **데이터 스냅샷:** `tools/export_dual_freqdae_data.py` 사용해 테스트 세트 `.npz` 보존 → 회귀 테스트 기준.
4. **회귀 테스트:** 주간 빌드 시 50개 기준 비트 추론 후 PRD / COS_SIM / SNR 임계치 통과 여부 자동화.
5. **로그 보관:** `history.json`, `metrics_summary.json`을 중앙 로그 서버 또는 MLflow에 기록 고려.

9. 자주 발생하는 문제 (Common Issues)

문제	증상	해결
입력 길이 오류	<code>ValueError: length != 512</code>	512 고정 길이로 재샘플/패딩 또는 <code>--segment</code> 사용
CUDA 플러그인 경고	<code>cuBLAS/cuDNN 이미 등록</code>	중복 설치 가능성 → 단일 TF 버전 유지, 무시 가능
첫 실행 매우 느림	Fourier/FFT 변환 캐시 없음	<code>--reuse-cache</code> 로 재사용, 캐시 파일 보존
학습 진동/수렴 불량	<code>val_loss</code> 반복 상승/하강	LR 감소, <code>patience</code> 증가, 손실 함수 교체(Huber)
메모리 부족(OOM)	GPU 메모리 에러	batch size 축소 또는 mixed precision 적용

10. 성능/지표 (Metrics)

평가 산출:

- **RMSE:** 재구성 오차 크기 직관적 판단
- **PRD:** 재구성 품질 상대 백분율 (낮을수록 좋음)
- **COS_SIM:** 형태 보존도 (높을수록 좋음)
- **SNR(dB):** 신호 대비 복원 잔차 노이즈 비율 개선 (높을수록 좋음)

지표 저장 위치: `metrics_summary.json`, `metrics_per_sample.csv`

권장 목표 범위(데이터/노이즈 조건 따라 변동):

Metric	양호 기준(예시)
PRD	< 40%
COS_SIM	> 0.9
SNR	> 10 dB
RMSE	도메인 특성(신호 정규화 방식)에 따라 상대 비교

11. 부록 (Appendix)

11.1 빠른 실행 명령 모음

```
# 학습
python tools/train_dual_freqdae.py --exp-dir experiments/run1 --data-prep-
```

```

samples 512 --reuse-cache

# 평가만
python tools/train_dual_freqdae.py --exp-dir experiments/run1 --evaluate-
only

# 드라이런
python tools/train_dual_freqdae.py --dry-run --exp-dir experiments/dry

# 데이터셋 내보내기
python tools/export_dual_freqdae_data.py --samples 512 --out-dir exported -
-reuse-cache

# 비트 추론
python tools/run_dual_freqdae.py --input exported/X_test.npy --weights
experiments/run1/Dual_FreqDAE_weights.best.weights.h5 --output denoised.npy

# 긴 신호 분할 추론
python tools/run_dual_freqdae.py --input long_signal.npy --segment --hop
512 --pad --weights experiments/run1/Dual_FreqDAE_weights.best.weights.h5 -
-output denoised_segments.npy

```

11.2 용어 정리

용어	설명
Beat	심장 전기 신호 한 박자 단위 (P-QRS-T 포함)
PRD	Percentage Root-mean-square Difference
COS_SIM	Cosine Similarity (형태 유사도)
SNR	Signal-to-Noise Ratio
FAN Layer	주기/위상 + gating을 활용한 확장 표현 레이어